

Day-71-NumPy Fundamentals — Arrays

1. Day Number

Day 71

2. Topic Name

NumPy Fundamentals — NumPy Arrays

Today you will learn how NumPy stores multiple numbers inside an **array**.

3. Connection to Previous Learning

You already know how to store multiple values using a normal Python list:

```
prices = [100, 200, 300, 400]
```

Today you will learn about **NumPy arrays**.

NumPy arrays are widely used in:

- Data science
- Machine learning
- Statistics
- Scientific computing
- Numerical calculations

The idea is similar to a Python list, but NumPy arrays are designed especially for working efficiently with numbers.

4. Important Topics

NumPy

NumPy stands for **Numerical Python**.

It is an external Python library used for numerical data.

We normally import it like this:

```
import numpy as np
```

np is simply a short name for NumPy.

If NumPy is not installed:

```
pip install numpy
```

Array

A NumPy **array** is a container that can hold multiple values.

Example:

```
numbers = np.array([5, 10, 15, 20])
```

Here:

```
5  
10  
15  
20
```

are stored inside one NumPy array.

Shape

The **shape** tells us the structure of an array.

For example, an array containing four values might have:

```
(4,)
```

The 4 means there are four elements.

The comma is part of Python's representation of a one-dimensional shape.

You can check it using:

```
numbers.shape
```

dtype

dtype means **data type**.

It tells NumPy what kind of values are stored inside the array.

For example, an array containing whole numbers may have a dtype similar to:

```
int64
```

An array containing decimals may have something similar to:

```
float64
```

The exact dtype can vary depending on your computer and environment.

You can check it using:

```
numbers.dtype
```

One-Dimensional Array

For now, you will work only with a **one-dimensional array**.

Example idea:

```
[10, 20, 30, 40]
```

Think of it as one straight row of values.

We will not introduce multidimensional arrays yet.

5. Foundational Notes

Creating an Array

NumPy provides:

```
np.array()
```

You can give it a Python list:

```
np.array([2, 4, 6])
```

NumPy converts those values into an array.

Accessing Values

Like Python lists, NumPy arrays use indexes.

```
numbers[0]
```

means:

```
first value
```

Remember that indexing starts at 0.

Checking the Number of Elements

Python's `len()` also works with a one-dimensional NumPy array:

```
len(numbers)
```

You can also inspect:

```
numbers.shape
```

The two give related information, but shape becomes especially useful later when arrays become more complex.

6. Python List vs NumPy Array

Suppose we have these values:

```
[10, 20, 30, 40]
```

A normal Python list might be created like this:

```
prices = [10, 20, 30, 40]
```

A NumPy array might be created like this:

```
prices = np.array([10, 20, 30, 40])
```

The main beginner-level difference is:

Python List	NumPy Array
Built into Python	Comes from NumPy
Can easily contain different types	Usually works best with one consistent data type
Good for general-purpose programming	Excellent for numerical calculations
Uses []	Often created with <code>np.array()</code>
Common in normal Python programs	Common in data science and ML

For example, Python allows:

```
data = [10, "Apple", True]
```

NumPy arrays are usually used more like:

```
10, 20, 30, 40
```

or:

```
10.5, 20.7, 30.2
```

where the values represent similar types of data.

7. Easy Example

Imagine that you recorded the number of products sold on four days:

```
12, 15, 10, 18
```

You could create a NumPy array for them:

```
sales = np.array([12, 15, 10, 18])
```

You could then inspect properties such as:

```
sales.shape
```

and:

```
sales.dtype
```

Conceptually:

```
sales
↓
[12 15 10 18]

shape
↓
(4,)

dtype
↓
integer type
```

Notice that when NumPy prints an array, it normally does not show commas between the values.

8. Problem Statement

Create a NumPy array containing **five product prices**.

For example, your program could work with prices such as:

```
120
250
80
```

175 300

Your program should:

1. Import NumPy.
2. Create one NumPy array containing five prices.
3. Print the complete array.
4. Print its length or shape.
5. Print its data type.

Keep the program small.

Do not perform advanced mathematical operations yet.

9. Concepts Used

For this exercise you will use:

NumPy library ↓ np.array() ↓ One-dimensional array ↓ len() ↓ .shape ↓ .dtype ↓ print()
--

You are combining concepts you already know—variables, numbers, functions, and printing—with your first NumPy-specific concepts.

10. Thought Process

Before writing code, think through the problem.

Step 1: What data do I have?

Five product prices.

120, 250, 80, 175, 300

Step 2: How should I store them?

Instead of a normal Python list, today's goal is to use a:

```
NumPy array
```

Step 3: What information must I display?

You need:

```
the array itself  
its size/shape  
its data type
```

Step 4: Which NumPy features help?

Think about:

```
np.array(...)  
.shape  
.dtype
```

You do not need loops or complicated functions for this exercise.

11. Beginner-Friendly Pseudocode

```
START  
  
import NumPy using the common short name  
  
create five product prices  
  
convert/store those prices as a NumPy array  
  
print the complete array  
  
find and print the array length or shape  
  
find and print the array data type  
  
END
```

A slightly more detailed version:

```
IMPORT NumPy  
  
CREATE prices array containing five numbers  
  
PRINT "Prices:"
```

```
PRINT prices

PRINT "Shape:"
PRINT shape of prices

PRINT "Data type:"
PRINT data type of prices
```

Translate these instructions into Python yourself.

12. Suggested Solving Approach — NumPy Approach

For this exercise, follow this flow:

```
Import NumPy
  ↓
Prepare five prices
  ↓
Create NumPy array
  ↓
Inspect the array
  ↓
Inspect shape
  ↓
Inspect dtype
```

Avoid converting everything back into normal Python lists. The goal is to become comfortable working directly with a NumPy array.

13. Easy Edge Cases

Edge Case 1: Empty Array

You might accidentally create:

```
np.array([])
```

There are no elements.

Its shape would indicate zero elements:

```
(0,)
```

Your program should still be able to print the array without crashing.

Edge Case 2: Decimal Values

Product prices often contain decimals.

For example:

```
99.50
125.75
200.00
```

If decimals are included, NumPy will normally use a floating-point data type instead of an integer data type.

Conceptually:

```
Whole numbers
100, 200, 300
      ↓
integer dtype
```

while:

```
Decimal numbers
99.5, 125.75, 200.0
      ↓
floating-point dtype
```

This is one reason checking `.dtype` is useful.

14. Expected Output

If your five prices are:

```
120, 250, 80, 175, 300
```

your output should look approximately like:

```
Product prices: [120 250  80 175 300]
Shape: (5,)
Data type: int64
```

You might see another integer type such as:

```
int32
```

instead of:

```
int64
```

That can be completely normal.

If you use decimal prices, the dtype may look similar to:

```
float64
```

The important idea is recognizing the difference between **integer** and **floating-point** array data.

15. Hint Only

Start with the standard NumPy import:

```
import numpy as np
```

Then remember these three NumPy ideas:

```
np.array(...)
```

```
your_array.shape
```

```
your_array.dtype
```

Your program only needs **one array containing five prices** and a few `print()` statements.

Do not look for a complicated solution—Day 71 is mainly about creating and inspecting your first NumPy array.